

Universidade
Estadual de Londrina

Marco Antônio Cottorello Henry

Murilo de Souza Neves

Rodrigo Macedo da Silva Junior

# VIRTUALIZAÇÃO, NUVEM E CONTÊINERES

Arquitetura, modelos de serviço, segurança e eficiência energética

LONDRINA - PR

2026

HENRY, Marco Antônio Cottorello. NEVES, Murilo de Souza. SILVA JUNIOR, Rodrigo Macedo da. **Virtualização, Nuvem e Contêineres**. 2026. 24 folhas. (Graduação em Ciência da Computação) – Universidade Estadual de Londrina, Londrina, 2026.

## RESUMO

A consolidação da computação em nuvem deslocou a infraestrutura de tecnologia da informação do servidor dedicado para recursos provisionados sob demanda, sustentados por camadas sucessivas de abstração. Este trabalho apresenta essa pilha de forma integrada, partindo da definição canônica do NIST, com as cinco características essenciais, os três modelos de serviço e os quatro modelos de implantação, e do critério que efetivamente separa IaaS, PaaS e SaaS, que é a posição da fronteira de controle sobre a pilha de software. Em seguida examina duas implementações situadas na camada de abstração e controle de recursos. A primeira é o OpenStack, cuja arquitetura modular e cujos serviços de identidade, imagem, computação e rede convertem servidores físicos em infraestrutura provisionável por interface programável. A segunda é o Docker, cuja engine decomposta, cujas primitivas de isolamento do kernel e cujos modelos de rede local definem o contêiner como processo isolado sobre kernel compartilhado. A partir dessa base, o trabalho analisa a superfície de ataque decorrente do compartilhamento de kernel, com os mecanismos de contenção de privilégios, as vulnerabilidades de imagens e a proteção do tráfego entre contêineres, e discute a eficiência energética de data centers, com a métrica PUE, a consolidação dinâmica de servidores e a gestão térmica. Conclui-se que as escolhas de isolamento e de densidade de consolidação são interdependentes, de modo que segurança e eficiência energética constituem dimensões acopladas de um mesmo projeto de infraestrutura.

**Palavras-chave:** Computação em Nuvem. Virtualização. Contêineres. OpenStack. Docker. Eficiência Energética.

HENRY, Marco Antônio Cottorello. NEVES, Murilo de Souza. SILVA JUNIOR, Rodrigo Macedo da. **Virtualization, Cloud and Containers**. 2026. 24 pages. (Graduação em Ciência da Computação) – State University of Londrina, Londrina, 2026.

## ABSTRACT

The consolidation of cloud computing has shifted information technology infrastructure from dedicated servers to resources provisioned on demand and supported by successive layers of abstraction. This work presents that stack in an integrated manner, starting from the canonical NIST definition, with its five essential characteristics, three service models and four deployment models, and from the criterion that actually separates IaaS, PaaS and SaaS, namely the position of the control boundary over the software stack. It then examines two implementations located in the resource abstraction and control layer. The first is OpenStack, whose modular architecture and whose identity, image, compute and networking services turn physical servers into infrastructure provisionable through a programmable interface. The second is Docker, whose decomposed engine, kernel isolation primitives and local network models define the container as an isolated process running on a shared kernel. On that basis, the work analyses the attack surface arising from kernel sharing, covering privilege containment mechanisms, image vulnerabilities and protection of inter-container traffic, and discusses energy efficiency in data centres, covering the PUE metric, dynamic server consolidation and thermal management. It concludes that isolation choices and consolidation density are interdependent, so that security and energy efficiency are coupled dimensions of a single infrastructure design.

**Key words:** Cloud Computing. Virtualization. Containers. OpenStack. Docker. Energy Efficiency.

# Lista de Figuras

## Lista de Tabelas

**2.1 Escopo de controle entre provedor e consumidor por modelo de serviço.** . . 10
**3.2 Serviços do OpenStack e atribuições declaradas na documentação oficial.** . 14
**4.3 Modelos de rede local do Docker.** . . . . . 16
**6.4 Síntese integradora entre segurança de contêineres e eficiência energética em Data Centers.** . . . . . 20

## LISTA DE ABREVIATURAS E SIGLAS

| | |
|---|---|
| AMQP | *Advanced Message Queuing Protocol* |
| API | *Application Programming Interface* (Interface de Programação de Aplicações) |
| ASHRAE | *American Society of Heating, Refrigerating and Air-Conditioning Engineers* |
| CCEE | Câmara de Comercialização de Energia Elétrica |
| CI/CD | *Continuous Integration / Continuous Delivery* (Integração Contínua e Entrega Contínua) |
| CPU | *Central Processing Unit* (Unidade Central de Processamento) |
| CVE | *Common Vulnerabilities and Exposures* |
| eBPF | *Extended Berkeley Packet Filter* |
| EPE | Empresa de Pesquisa Energética |
| FaaS | *Function as a Service* (Função como Serviço) |
| IaaS | *Infrastructure as a Service* (Infraestrutura como Serviço) |
| IEA | *International Energy Agency* (Agência Internacional de Energia) |
| IP | *Internet Protocol* (Protocolo de Internet) |
| ISO | *International Organization for Standardization* |
| LBNL | *Lawrence Berkeley National Laboratory* |
| NIST | *National Institute of Standards and Technology* |
| OCI | *Open Container Initiative* |
| ONS | Operador Nacional do Sistema Elétrico |
| PaaS | *Platform as a Service* (Plataforma como Serviço) |
| PID | *Process Identifier* (Identificador de Processo) |
| PUE | *Power Usage Effectiveness* (Eficácia do Uso de Energia) |
| RAM | *Random Access Memory* (Memória de Acesso Aleatório) |
| REST | *Representational State Transfer* |
| SaaS | *Software as a Service* (Software como Serviço) |
| SIN | Sistema Interligado Nacional |
| SLA | *Service Level Agreement* (Acordo de Nível de Serviço) |
| TI | Tecnologia da Informação |
| TWh | Terawatt-hora |
| VM | *Virtual Machine* (Máquina Virtual) |

**Sumário**


**1 Introdução** **7**

**2 Computação em Nuvem** **8**

2.1 MODELOS DE SERVIÇO . . . . . 9

2.2 MODELOS DE IMPLANTAÇÃO . . . . . 10

2.3 ARQUITETURA DE REFERÊNCIA E CAMADAS DE ORQUESTRAÇÃO . . . . . 11

**3 Arquitetura Modular do OpenStack e Gerenciamento de IaaS** **13**

**4 Engine, Isolamento e Redes Locais no Docker** **15**

**5 Segurança no Ambiente de Cloud e Contêineres** **17**

**6 Eficiência energética** **19**

**REFERÊNCIAS** **22**

# 1 INTRODUÇÃO

# 2 COMPUTAÇÃO EM NUVEM

A computação em nuvem é definida pelo Instituto Nacional de Padrões e Tecnologia dos Estados Unidos (NIST) como um modelo que habilita acesso ubíquo, conveniente e sob demanda, por meio da rede, a um conjunto compartilhado de recursos computacionais configuráveis, redes, servidores, armazenamento, aplicações e serviços, que podem ser rapidamente provisionados e liberados com esforço mínimo de gerenciamento ou de interação com o provedor de serviço [1]. A definição propõe permitir comparações amplas entre serviços e estratégias de implantação, sem restringir métodos particulares de implantação, entrega de serviço ou operação de negócio. O modelo é composto por cinco características essenciais, três modelos de serviço e quatro modelos de implantação.

As cinco características essenciais delimitam o que caracteriza um sistema como nuvem e devem ser satisfeitas conjuntamente. O *autoatendimento sob demanda* estabelece que o consumidor provisione capacidades computacionais, como tempo de servidor e armazenamento em rede, unilateral e automaticamente, sem requerer interação humana. O *amplo acesso à rede* exige que as capacidades estejam disponíveis pela rede e sejam acessadas por mecanismos padronizados, utilizáveis por diferentes tipos de plataformas clientes. O *agrupamento de recursos* determina que os recursos do provedor sejam agrupados para servir múltiplos consumidores em um modelo de múltiplos inquilinos, com recursos físicos e virtuais atribuídos e reatribuídos dinamicamente conforme a demanda, havendo independência de localização no sentido de que o consumidor geralmente desconhece a localização exata dos recursos, podendo especificá-la apenas em nível mais alto de abstração, como país, estado ou *data center*. A *elasticidade rápida* consiste na capacidade de liberar recursos elasticamente, em alguns casos de forma automática, escalando horizontalmente ou verticalmente de acordo com à demanda, de modo que as capacidades disponíveis aparentem ser ilimitadas ao consumidor. O *serviço medido* requer que o sistema controle e otimize automaticamente o uso de recursos por meio de uma capacidade de medição apropriada ao tipo de serviço, com uso monitorado, controlado e reportado, tipicamente em base de pagamento por uso [1].

A distinção entre elasticidade e escalabilidade é relevante para a correta classificação dos sistemas. Escalabilidade designa a manutenção do atendimento adequado conforme a carga aumenta, enquanto elasticidade designa a variação rápida da capacidade alocada, para mais e para menos, com liberação dos recursos que não estão sendo utilizados.

## 2.1 MODELOS DE SERVIÇO

Os três modelos de serviço são definidos pela capacidade fornecida ao consumidor. Em *Infrastructure as a Service* (IaaS), a capacidade consiste em provisionar processamento, armazenamento, redes e outros recursos computacionais fundamentais, nos quais o consumidor implanta e executa software arbitrário, incluindo sistemas operacionais e aplicações. Com outras palavras, o consumidor não gerencia a infraestrutura subjacente, mas controla sistemas operacionais, armazenamento e aplicações implantadas, além de possivelmente dispor de controle limitado sobre componentes de rede selecionados. Em *Platform as a Service* (PaaS), a capacidade consiste em implantar sobre a infraestrutura de nuvem aplicações criadas ou adquiridas pelo consumidor, construídas com linguagens de programação, bibliotecas, serviços e ferramentas suportadas pelo provedor. O consumidor controla as aplicações implantadas e, possivelmente, configurações do ambiente de hospedagem, sem controle sobre rede, servidores, sistemas operacionais ou armazenamento. Em *Software as a Service* (SaaS), a capacidade consiste em utilizar as aplicações do provedor executando em uma infraestrutura de nuvem, acessíveis por interface de cliente leve, como um navegador, ou por interface de programa, restando ao consumidor, no máximo, configurações limitadas da aplicação específicas do usuário [1].

O critério que separa os três modelos é a fronteira de controle sobre o software. A arquitetura de referência do NIST formaliza essa delimitação por meio das camadas de aplicação, de *middleware* e de sistema operacional, atribuindo a cada modelo de serviço a parte que cabe ao consumidor e a que cabe ao provedor [2]. A camada de aplicação é utilizada pelos consumidores de SaaS e instalada, gerenciada e mantida pelos consumidores de PaaS e de IaaS e pelos provedores de SaaS. A camada de *middleware*, que fornece os blocos de construção de software como bibliotecas e bancos de dados, é utilizada pelos consumidores de PaaS, mantida pelos consumidores de IaaS ou pelos provedores de PaaS, e oculta dos consumidores de SaaS. A camada de sistema operacional é oculta dos consumidores de SaaS e de PaaS; em IaaS, o consumidor assume responsabilidade integral pelos sistemas operacionais hóspedes, enquanto o provedor controla o sistema operacional anfitrião e os hipervisores de virtualização. A Tabela 2.1 sintetiza essa distribuição.

**Tabela 2.1** – Escopo de controle entre provedor e consumidor por modelo de serviço.

| **Camada** | **SaaS** | **PaaS** | **IaaS** |
|---|---|---|---|
| Aplicação | Utilizada pelo consumidor; mantida pelo provedor | Instalada e mantida pelo consumidor | Instalada e mantida pelo consumidor |
| *Middleware* | Oculta do consumidor | Utilizada pelo consumidor; mantida pelo provedor | Instalada e mantida pelo consumidor |
| Sistema operacional | Oculta do consumidor | Oculta do consumidor | Hóspede sob responsabilidade do consumidor; anfitrião sob controle do provedor |
| Hardware e hipervisor | Provedor | Provedor | Provedor |

A delimitação de controles tem consequência direta sobre a atribuição de responsabilidades de segurança, uma vez que cada parte responde pela camada que efetivamente controla [2]. Em IaaS, a correção de vulnerabilidades do sistema operacional hóspede permanece com o consumidor. Já em arquiteturas PaaS e SaaS, essa atribuição transfere-se ao provedor, ao mesmo tempo que o consumidor perde a possibilidade de inspecionar e configurar as camadas inferiores. Em IaaS, máquinas virtuais distintas compartilham o hardware por meio de um hipervisor. Em PaaS, processos distintos compartilham o sistema operacional e os serviços de suporte. Já em SaaS, consumidores distintos podem compartilhar a mesma aplicação ou o mesmo banco de dados [3]. Como os mecanismos de compartilhamento dependem de utilitários complexos para manter isoladas as cargas de trabalho, existe o risco de falha de isolamento, que pode ser diminuido pela cifragem dos dados antes da inserção na nuvem, pela limitação dos tipos de dados processados ou pela contratação de mecanismos de isolamento especializados, como o aluguel de sistemas computacionais inteiros em lugar de máquinas virtuais, redes privadas virtuais e redes segmentadas [3].

## 2.2 Modelos de implantação

Os quatro modelos de implantação especificam para quem a infraestrutura é liberada, e suas diferenças baseiam-se no grau de exclusividade com que os recursos são disponibilizados a um consumidor [2]. A *nuvem privada* é provisionada para uso exclusivo de uma única organização composta por múltiplos consumidores, podendo ser possuída, gerenciada e operada pela organização, por um terceiro ou por combinação de ambos, e podendo existir dentro ou fora de suas instalações. A *nuvem comunitária* é provisionada para uso exclusivo de uma comunidade específica de consumidores, pertencentes a organizações com preocupações compartilhadas, tais como missão, requisitos de segurança, política e conformidade. A *nuvem pública* é provisionada para uso aberto ao público em geral e existe nas instalações do provedor. A *nuvem híbrida* é a composição de duas ou mais infraestruturas

distintas que permanecem entidades únicas, ligadas por tecnologia padronizada ou proprietária que habilita a portabilidade de dados e de aplicações, tendo como exemplo o *cloud bursting* para balanceamento de carga entre nuvens [1].

Cabe observar que o critério dos modelos é a exclusividade de uso, e não a localização física da infraestrutura. A definição de nuvem privada admite expressamente a operação por terceiro e a existência fora das instalações da organização, razão pela qual a arquitetura de referência distingue as variantes privada *on-site* e privada terceirizada, com variantes análogas para a nuvem comunitária, apenas a nuvem pública tem a localização fixada pela definição [2]. A escolha do modelo condiciona simultaneamente os riscos de múltiplos inquilinos e o teto de elasticidade, a restrição do conjunto de inquilinos possíveis, nas variantes *on-site*, mitiga parte dos riscos de isolamento, ao custo de uma capacidade fixa em cada instante, limitada pelos recursos do próprio *data center*. A nuvem pública, não restrita quanto a localização ou tamanho e capaz de empregar múltiplos inquilinos sem limitação por perímetros de segurança estáticos, apresenta vantagem na obtenção de elasticidade, ou da aparência de disponibilidade ilimitada de recursos [3].

## 2.3 Arquitetura de referência e camadas de orquestração

A arquitetura de referência do NIST define cinco atores, descritos a seguir.

- *cloud consumer*, responsável por manter a relação de negócio com o provedor e por utilizar o serviço.
- *cloud provider*, responsável por tornar o serviço disponível.
- *cloud carrier*, intermediário que fornece conectividade e transporte do serviço.
- *cloud auditor*, responsável pela avaliação independente de desempenho, segurança e conformidade.
- *cloud broker*, entidade que gerencia uso, desempenho e entrega de serviços e negocia as relações entre provedores e consumidores [2].

Os acordos de nível de serviço (SLAs) firmados entre consumidor e provedor especificam os requisitos técnicos de desempenho a serem cumpridos, podendo também relacionar as promessas explicitamente não oferecidas e as obrigações atribuídas ao consumidor. Quando há intermediação de transporte, o provedor estabelece acordos distintos com o *carrier* e com o consumidor, podendo exigir do primeiro conexões dedicadas e cifradas para atender às obrigações assumidas perante o segundo.

As atividades do provedor relativas à composição dos recursos são modeladas em três camadas de orquestração de serviço, nas quais o empilhamento representa dependência [2]. A *camada de serviço* contém as interfaces de acesso dos consumidores, nela se situando as interfaces dos três modelos de serviço. A *camada de abstração e controle de recursos* contém os componentes que fornecem e gerenciam o acesso aos recursos físicos por meio de abstração de software, tais como hipervisores, máquinas virtuais e armazenamento virtual de dados, bem como os componentes de controle responsáveis pela alocação de recursos, pelo controle de acesso e pelo monitoramento de uso; é esse conjunto de software que habilita o agrupamento de recursos, a alocação dinâmica e o serviço medido. A *camada de recursos físicos* compreende os recursos de hardware, como computadores, redes e armazenamento, e os recursos de instalação, tais como climatização, energia e comunicações. Os consumidores acessam a camada de serviço e não dispõem de acesso direto aos recursos físicos.

É nessa camada intermediária que se insere a virtualização completa, na qual um ou mais sistemas operacionais e suas aplicações executam sobre hardware virtual, cada instância em uma máquina virtual gerenciada por um hipervisor que controla o fluxo de instruções entre os hóspedes e o hardware físico, particionando os recursos e isolando os hóspedes [4]. O NIST registra, ainda, que a virtualização de máquina é a tecnologia comumente empregada nessa camada, mas que outros meios de fornecer as abstrações de software necessárias também são possíveis [2], posição que acomoda a virtualização em nível de sistema operacional por contêineres discutida nas seções seguintes. Cabe notar que a dependência entre os modelos de serviço é opcional, pois é possível, porém não necessário, que aplicações SaaS sejam construídas sobre componentes PaaS e que estes sejam construídos sobre componentes IaaS, podendo cada componente de serviço existir isoladamente [2].

A taxonomia estabelecida em 2011 não contempla, por fim, modelos de serviço posteriores. Serviços de função como serviço (*Function as a Service*) e computação *serverless* não dispõem de modelo próprio na definição e, pelos critérios nela adotados, classificam-se como PaaS, dado que o consumidor implanta código enquanto o provedor fornece o ambiente de execução sem expor sistema operacional ou servidores. Os contêineres, igualmente, não constituem modelo de serviço, situando-se como mecanismo da camada de abstração e controle de recursos, condição que fundamenta o tratamento dado a eles nas seções subsequentes, dedicadas à segurança do isolamento e à eficiência energética da consolidação de cargas.

# 3 ARQUITETURA MODULAR DO OPENSTACK E GERENCIAMENTO DE IAAS

OpenStack é uma plataforma de código aberto para a construção de nuvens de infraestrutura, composta por um conjunto de serviços independentes que cooperam. Na arquitetura de referência discutida anteriormente, ocupa a camada de abstração e controle de recursos, na qual fornece as abstrações de software sobre os recursos físicos, como máquinas virtuais, redes virtuais, imagens e volumes, bem como os componentes de controle responsáveis pela alocação de recursos, pelo controle de acesso e pelo monitoramento de uso, expondo na camada de serviço as interfaces pelas quais o consumidor provisiona processamento, armazenamento e redes [2]. Constitui, portanto, implementação concreta do modelo de serviço *Infrastructure as a Service* [1].

A documentação oficial estabelece um padrão interno comum a todos os serviços da plataforma [5]. Cada serviço possui ao menos um processo de API, que "escuta requisições de API, pré-processa-as e as repassa a outras partes do serviço"; com exceção do serviço de identidade, "o trabalho efetivo é realizado por processos distintos". O estado de cada serviço reside em banco de dados próprio, e para a comunicação entre os processos de um mesmo serviço "um *broker* de mensagens AMQP é utilizado", sendo o RabbitMQ a escolha corrente. Já a comunicação entre serviços distintos obedece a outro regime, pois "serviços individuais interagem entre si por meio de APIs públicas, exceto onde comandos administrativos privilegiados são necessários". Dessa separação decorre o acoplamento fraco entre serviços, que podem ser substituídos, escalados ou atualizados de forma independente desde que preservem o contrato de suas APIs. A autenticação, por sua vez, é centralizada, pois "todos os serviços autenticam-se por meio de um serviço de identidade comum". O acesso à plataforma admite o painel web Horizon, clientes de linha de comando, requisições diretas de API e bibliotecas de desenvolvimento, mas a documentação ressalva que "todos esses métodos de acesso emitem chamadas de API REST aos diversos serviços do OpenStack", de modo que painel e linha de comando são clientes da mesma interface, e não caminhos administrativos alternativos.

Os quatro serviços principais possuem atribuições declaradas de forma precisa, sintetizadas na Tabela 3.2 [5]. Cabe destacar dois pontos. O primeiro é que a autorização do Keystone se estrutura sobre projetos, usuários, papéis e domínios, e é a atribuição de papéis a usuários dentro de projetos que delimita quais recursos de quais inquilinos cada credencial alcança. O segundo é que o Nova, implementado como "um conjunto de *daemons* sobre servidores Linux existentes", admite não apenas máquinas virtuais, mas também servidores em *baremetal* e contêineres de sistema, mantendo arquitetura unificada entre múltiplos *drivers* de computação.

A articulação desses serviços é o que converte servidores físicos em infraestrutura provisionável sob demanda. A documentação do Nova declara quatro

**Tabela 3.2** – Serviços do OpenStack e atribuições declaradas na documentação oficial.

| **Serviço** | **Função** | **Atribuição** |
|---|---|---|
| Keystone | Identidade | Autenticação de clientes de API, descoberta de serviços e autorização multi-inquilino sobre projetos, usuários, papéis e domínios |
| Glance | Imagem | Descoberta, registro e recuperação de imagens de máquina virtual, com metadados por API REST |
| Nova | Computação | Provisionamento de instâncias de computação como conjunto de *daemons* sobre servidores Linux |
| Neutron | Rede | Conectividade de rede como serviço entre dispositivos de interface gerenciados por outros serviços |
| Placement | Inventário | Rastreamento do inventário de recursos e escolha do provedor que atenderá à criação da instância |

dependências obrigatórias, das quais decorre a sequência de criação de uma instância, na qual a requisição é autenticada pelo Keystone, que "fornece identidade e autenticação para todos os serviços OpenStack"; a imagem de origem é obtida do Glance, pois "todas as instâncias de computação são iniciadas a partir de imagens do glance"; o Placement, "responsável por rastrear o inventário de recursos disponíveis em uma nuvem", auxilia "na escolha de qual provedor desses recursos será usado"; e o Neutron provisiona "as redes virtuais ou físicas às quais as instâncias de computação se conectam na inicialização". Cada serviço indisponível produz, assim, uma falha identificável, pois sem Keystone a requisição é recusada antes de qualquer alocação; sem Glance não há sistema de arquivos inicial; sem Placement não há informação para escolher o nó; sem Neutron a instância existe sem conectividade.

A correspondência com o modelo IaaS é direta. O operador controla o hardware, os hipervisores e os próprios serviços da plataforma, enquanto o consumidor controla o sistema operacional hóspede, o armazenamento atribuído e as aplicações que implanta, dispondo ainda do controle limitado de componentes de rede que a definição do modelo prevê, exercido pelas abstrações do Neutron [1, 2]. As características essenciais da nuvem encontram implementação identificável, uma vez que o autoatendimento sob demanda corresponde às APIs REST dos serviços; o agrupamento de recursos e a alocação dinâmica, ao inventário do Placement e à atribuição de recursos pelos *drivers* do Nova; a autorização multi-inquilino, ao modelo de projetos e papéis do Keystone. A plataforma não substitui a camada de virtualização, mas a coordena, uma vez que o Nova delega a execução das instâncias a *drivers* de computação, entre os quais hipervisores. Situa-se, assim, acima da virtualização e abaixo das interfaces consumidas pelo usuário, o que a distingue da conteinerização tratada na seção seguinte, uma vez que o OpenStack gerencia a oferta de recursos, enquanto os contêineres empacotam e isolam as cargas que executam sobre eles.

# 4 ENGINE, ISOLAMENTO E REDES LOCAIS NO DOCKER

O Docker é a plataforma de conteinerização mais difundida para a virtualização em nível de sistema operacional. Diferentemente da virtualização completa, os contêineres compartilham o *kernel* do hospedeiro e se distinguem entre si apenas pelos mecanismos de isolamento que esse *kernel* oferece [6].

A arquitetura é do tipo cliente-servidor, no qual o cliente traduz comandos em requisições a uma API REST, transmitidas por *socket* UNIX local ou por interface de rede, e o daemon `dockerd` as atende, gerenciando imagens, contêineres, redes e volumes; todo o estado reside no daemon [7]. A *engine*, porém, não é monolítica. O `dockerd` responde pela API de alto nível, o `containerd` pelo ciclo de vida dos contêineres e pelo armazenamento em camadas, e o *runtime* `runc` cria o processo isolado, configurando *namespaces*, aplicando limites de recursos e substituindo a si mesmo pelo processo da aplicação [8]. Entre o `containerd` e cada contêiner interpõe-se um *shim*, que permanece como processo pai e reporta o código de término, razão pela qual o daemon pode ser reiniciado sem derrubar os contêineres em execução [7]. Especificações abertas da *Open Container Initiative* sustentam essa decomposição, separando o formato da imagem da forma de executar um pacote já extraído, do que decorre a interoperabilidade entre *runtimes* compatíveis [8]. No armazenamento, a *engine* empilha as camadas imutáveis da imagem por um sistema de arquivos de união, tipicamente o *OverlayFS*, e acrescenta no topo uma camada de leitura e escrita exclusiva do contêiner, com semântica de cópia em escrita; daí decorrem o compartilhamento de camadas base entre imagens e o descarte das alterações não persistidas em volumes [7].

O isolamento não é produzido pelo Docker, e sim por duas famílias de primitivas do *kernel* Linux que a *engine* combina. Os *namespaces* determinam o que um processo enxerga, e os grupos de controle, ou *cgroups*, determinam quanto de recurso físico ele consome. Cada *namespace* particiona um recurso global, de modo que os processos nele contidos observem uma instância própria desse recurso. O de identificadores de processos atribui ao processo inicial o identificador 1 e impede a sinalização de processos do hospedeiro; o de montagem dá ao contêiner árvore própria de pontos de montagem, condição para que a raiz observada seja a visão unificada das camadas da imagem; o de rede fornece pilha de rede independente, base de todos os modelos discutidos adiante; os demais isolam a comunicação entre processos, o nome de máquina e o mapeamento de identificadores de usuário. Os *cgroups* contabilizam e limitam o consumo por grupo de processos mediante controladores de processador, memória, entrada e saída em disco e número de processos, contendo a exaustão de recursos do nó [6]. O *kernel*, contudo, permanece compartilhado e a interface de chamadas de sistema continua sendo superfície comum a todos os contêineres, razão pela qual são necessárias camadas adicionais de contenção, tratadas na seção seguinte. *Namespaces* e *cgroups* constituem mecanismo de isolamento, não fronteira de segurança equivalente à de uma máquina virtual [6].

A conectividade decorre do *namespace* de rede, e os modelos disponíveis, sintetizados na Tabela 4.3, correspondem aos tratamentos possíveis desse espaço [7]. No modelo *bridge*, que é o padrão, a *engine* cria no hospedeiro uma ponte em software e, para cada contêiner, um par de interfaces virtuais com uma extremidade no *namespace* do contêiner e a outra subordinada à ponte. O tráfego destinado a redes externas é submetido a mascaramento, de forma que dispositivos externos observem apenas o endereço do hospedeiro, enquanto a exposição no sentido inverso exige publicação explícita de porta; essa assimetria explica por que um contêiner alcança a Internet sem configuração, mas não é alcançável de fora sem publicação. Na rede *bridge* padrão os contêineres se comunicam apenas por endereço IP, ao passo que nas redes definidas pelo usuário um servidor de nomes interno resolve os nomes dos contêineres conectados, prática indispensável quando a aplicação é composta por múltiplos contêineres cujos endereços variam a cada reinício. Todos os modelos operam em um único hospedeiro, pois a comunicação entre contêineres distribuídos por múltiplos nós requer redes de sobreposição, pertencentes ao domínio da orquestração em *cluster*.

**Tabela 4.3** – Modelos de rede local do Docker.

| **Modelo** | ***Namespace* de rede** | **Resolução de nomes** | **Exposição ao exterior** |
|---|---|---|---|
| *Bridge* padrão | Próprio, ligado à ponte `docker0` | Apenas por endereço IP | Mascaramento; publicação de porta |
| *Bridge* do usuário | Próprio, ligado a ponte dedicada | Servidor de nomes interno | Mascaramento; publicação de porta |
| *Host* | Compartilhado com o hospedeiro | Do próprio hospedeiro | Direta, na porta real do hospedeiro |
| *None* | Próprio, apenas *loopback* | Inexistente | Nenhuma |
| *Macvlan* e *ipvlan* | Próprio, exposto à rede física | Conforme a infraestrutura | Direta, como dispositivo na rede local |

O isolamento sobre *kernel* compartilhado é o ponto de partida da análise de segurança apresentada a seguir, e a densidade de consolidação que os contêineres permitem é a premissa da análise de eficiência energética.

# 5 SEGURANÇA NO AMBIENTE DE CLOUD E CONTÊINERES

O uso de computação em nuvem e a criação de cada vez mais aplicações *cloud-native*, apoiado pela crescente preferência por modelos de arquitetura distribuída, principalmente os microsserviços são suportadas por aplicações por meio da transição de máquinas virtuais baseadas em hipervisores[1] para a virtualização em nível de sistema operacional via contêineres.

Enquanto na virtualização tradicional cada instância executa um sistema operacional convidado completo sobre um hipervisor sobre o hardware físico, os contêineres compartilham diretamente o mesmo *kernel* do sistema hospedeiro [9]. Embora essa arquitetura reduza drasticamente o *overhead* de memória e ciclos de processamento, ela aumenta a possibilidade de ataques em um sistema que contenha ambientes de múltiplos inquilinos (nesse caso, os contêineres). Uma vulnerabilidade na interface de chamadas de sistema ou uma falha lógica no *kernel* compartilhado pode ser explorada para romper as da instância, resultando em ataques de escape de contêiner, nos quais um processo malicioso obtém acesso irrestrito ao sistema hospedeiro [6].

Para controlar os privilégios desses contêineres e hipervisores, é necessário realizar um isolamento em nível de sistema operacional, isso é feito por meio de primitivas de contenção do *kernel*. Por meio de *namespaces*, é possível isolar a visibilidade particionando recursos globais como árvores de identificadores de processos (*PID*), pilhas de rede, pontos de montagem e identificadores de usuários, de modo que cada contêiner enxergue apenas seu próprio escopo operacional [8]. Também é possível limitar o uso de recursos físicos de uma máquina por meio dos grupos de controle. Eles atuam na contabilização e limitação do consumo de recursos físicos como tempo de CPU, memória RAM e taxa de entrada/saída, prevenindo que uma instância comprometida execute ataques de negação de serviço por exaustão de hardware no nó físico.

Contudo, como os *namespaces* e groupos de controle não filtram a totalidade das interações com o kernel, a defesa em profundidade exige a adoção de contêineres sem privilégios de superusuário (*rootless containers*), a remoção de privilégios administrativos mediante a fragmentação de *Linux Capabilities* (descartando permissões críticas como *cap_sys_admin* [2]), a restrição de chamadas de sistema via perfis Seccomp (*Secure Computing Mode*) e o confinamento por Controle de Acesso Obrigatório [8].

A segurança em ambientes conteinerizados também é fortemente impactada por vulnerabilidades nas imagens desses e na cadeia de suprimentos de software. Imagens de contêineres são frequentemente derivadas de repositórios públicos não auditados, muitas

[1] Um hipervisor é um programa que acrescenta recursos de computação, como processamento, memória e armazenamento, e os redireciona entre máquinas virtuais (VMs)

[2] funciona essencialmente como um equivalente ao acesso tradicional de superusuário (*root*) para uma vasta gama de operações do sistema.

dessas imagens contêm bibliotecas de software desatualizadas, pacotes com falhas de segurança conhecidas não corrigidas ou até software malicioso [10]. A mitigação desse risco requer varreduras automatizadas de vulnerabilidades diretamente nas esteiras de integração contínua (CI/CD), a assinatura criptográfica de artefatos para validação de proveniência no *registry* e a adoção de imagens base mínimas ou *distroless*. Essas imagens teiram apenas a aplicação contida no contêiner e suas dependências, eliminando interpretadores *shell*, e gerenciadores de pacotes, que podem ser instrumentalizados por atacantes [6].

Como abordado anteriormente, atualmente a maior parte dos sistemas de grande e médio porte opta por uma arquitetura de microsserviços, que expande exponencialmente o volume de tráfego interno entre contêineres, denominado tráfego Leste-Oeste . Por padrão, orquestradores priorizam a conectividade plana (*flat networking*), na qual qualquer contêiner (ou *Pod*, conjunto de um ou mais contêineres) pode se comunicar livremente com outro endereço IP virtual do *cluster* sem restrições nativas de camada de rede [11]. Essa topologia facilita a movimentação de um invasor após o comprometimento inicial de um único microsserviço exposto. A neutralização desse vetor fundamenta-se na arquitetura *Zero Trust*, implementada por meio de microssegmentação lógica via políticas de rede do tipo negação por padrão (*default-deny*), criptografia e autenticação mútua na camada de transporte gerenciada por malhas de serviço, e inspeção de pacotes de alta performance diretamente no espaço de kernel utilizando Extended Berkeley Packet Filter (eBPF) [11].

A criticidade prática desses vetores de ataque e da contenção de recursos evidenciou-se em incidentes recentes na infraestrutura do *GitHub Actions*, cujos ambientes de integração contínua operam sobre *runners* efêmeros e contêineres isolados em nuvem. No âmbito da cadeia de suprimentos e isolamento de privilégios, o comprometimento da ação *tj-actions/changed-files* (CVE-2025-30066), em março de 2025, demonstrou como a ausência de restrições estritas de chamadas de sistema permite que códigos maliciosos inspecionem diretamente a memória do processo hospedeiro (*/proc/PID/mem*) para extrair credenciais de *registries* e chaves de nuvem em mais de 23 mil repositórios [12]. De forma análoga, a campanha *GhostAction*, documentada entre 2025 e 2026, explorou a ausência de microssegmentação e filtragem de tráfego de saída em contêineres de *build* para obter mais de 3.300 segredos corporativos em cerca de 30 mil repositórios [13]. Adicionalmente, do ponto de vista da disponibilidade e do isolamento de recursos via *cgroups*, a ocorrência de 48 interrupções críticas na plataforma ao longo de doze meses ilustra o impacto de falhas em cascata quando o particionamento de capacidade em ambientes de múltiplos inquilinos acaba falhando [14].

# 6 EFICIÊNCIA ENERGÉTICA

Com o crescimento da demanda por serviços em nuvem e processamento distribuído, o consumo e eficiência de Data Centers se mostrou como um desafio de engenharia. Para quantificar e padronizar a eficiência da infraestrutura física que abriga esses sistemas, o consórcio The Green Grid e a norma internacional ISO/IEC 30134-2 estabeleceram a métrica Power Usage Effectiveness (PUE) como o principal indicador de desempenho energético do setor [15]. O PUE é definido pela razão entre a energia total consumida pela instalação e a energia efetivamente entregue aos equipamentos de Tecnologia da Informação:

$$\text{PUE} = \frac{P_{\text{Total da Instalação}}}{P_{\text{Equipamentos de TI}}} = \frac{P_{\text{TI}} + P_{\text{Refrigeração}} + P_{\text{Perdas Elétricas}} + P_{\text{Iluminação/Auxiliares}}}{P_{\text{TI}}} \tag{1}$$

Em um cenário ideal, em que toda a energia fornecida ao edifício fosse integralmente consumida por servidores, ativos de armazenamento e comutadores de rede, o índice PUE seria igual a 1,0. Embora instalações de grande escala alcancem valores próximos a 1,1 ou 1,2 (ou seja, a instalação completa custa mais que somente os equipamentos de TI), a métrica apresenta uma limitação: o PUE avalia exclusivamente a eficiência da infraestrutura de suporte predial (climatização e conversão elétrica), mas não mensura a eficiência computacional interna dos servidores [16].

Essa distinção torna-se evidente ao analisar o fenômeno da não proporcionalidade energética do hardware de servidor. Servidores físicos operando em estado ocioso ou sob baixas taxas de utilização de CPU (entre 10% e 20%) ainda consomem cerca de 50% a 60% de sua potência elétrica máxima nominal devido à energização contínua de barramentos, memórias e fontes de alimentação [17]. Para eliminar esse desperdício de potência ociosa, Data Centers empregam algoritmos de consolidação dinâmica de servidores. Utilizando heurísticas de empacotamento aplicadas a máquinas virtuais e orquestradores de contêineres, as cargas de trabalho dispersas são migradas e concentradas no menor número possível de nós físicos ativos, permitindo que os servidores excedentes sejam colocados em estados de suspensão ou desligados, utilizando o recurso da elasticidade segundo a demanda. Porém, abaixar o denominador da equação pode acabar aumentando o valor total do PUE se a climatização e auxiliares não forem modulados junto com os servidores.

Consequentemente, a gestão térmica atua como o pilar complementar indispensável da eficiência energética, visto que os sistemas de refrigeração representam historicamente a maior parcela de consumo não-TI de um Data Center. A consolidação de cargas via contêineres eleva a utilização média das CPUs e intensifica a densidade de dissipação térmica por gabinete, exigindo arquiteturas de rejeição de calor alinhadas às diretrizes ambientais do comitê técnico ASHRAE TC 9.9 [18]. A otimização térmica moderna

estrutura-se em três pontos: (i) o isolamento aerodinâmico por meio da contenção de corredores frios ou quentes, que impede a recirculação e a mistura entre o ar insuflado e o ar de exaustão, permitindo elevar os pontos de ajuste de temperatura de entrada sem risco de formação de pontos quentes; (ii) a adoção de sistemas de *Free Cooling* (economizadores a ar ou a água), que aproveitam condições climáticas externas favoráveis para resfriar o fluido térmico sem o acionamento contínuo de compressores mecânicos; e (iii) o resfriamento líquido direto no componente (*Direct-to-Chip Liquid Cooling*) ou por imersão, cuja capacidade térmica volumétrica superior à do ar viabiliza a operação eficiente de racks de alta densidade [18]. A Tabela 6.4 sintetiza a correlação entre os mecanismos de segurança em contêineres e a eficiência energética da instalação.

**Tabela 6.4** – Síntese integradora entre segurança de contêineres e eficiência energética em Data Centers.

| **Dimensão** | **Desafio Técnico** | **Mecanismo / Solução** | **Impacto Cruzado (Segurança $\times$ Energia)** |
|---|---|---|---|
| Isolamento de Kernel | Superfície de ataque e escalada de privilégios | *Namespaces*, *cgroups*, *Seccomp* e *Rootless* | Viabiliza alta densidade de consolidação sem o gasto energético de VMs completas. |
| Imagens e Rede | CVEs em camadas base e tráfego *East-West* | Imagens *Distroless*, *Network Policies* e *eBPF* | Binários enxutos e filtragem no *kernel* via *eBPF* reduzem ciclos de CPU e rede. |
| Alocação de Nós | Desperdício por potência ociosa (*Idle Power*) | Consolidação dinâmica (*Bin Packing*) de cargas | Reduz a potência total de TI ($P_{\mathrm{TI}}$), exigindo contenção estrita de *multitenancy*. |
| Gestão Térmica | Dissipação de calor em *racks* densos e PUE alto | Contenção de corredores, *Free* e *Liquid Cooling* | Reduz a potência de refrigeração no numerador do PUE sob alta consolidação. |

A urgência em otimizar o PUE, consolidar servidores e modernizar a gestão térmica é amplificada pela recente expansão global de *data centers* de hiperescala, impulsionada pela computação em nuvem e por cargas intensivas de inteligência artificial. Relatórios da Agência Internacional de Energia (IEA) projetam que o consumo elétrico global do setor atingirá 945 TWh até 2030, um volume equivalente à demanda total de eletricidade do Japão, enquanto em polos consolidados, como a Irlanda, essas instalações já absorveram 23% de toda a eletricidade medida no país em 2025 [19, 20]. Nos Estados Unidos, levantamentos do *Lawrence Berkeley National Laboratory* (LBNL) apontam um consumo de 192 TWh em 2024 (4,7% da matriz elétrica nacional), com projeção de ascensão para 649 TWh até 2030 (11,8% da demanda nacional); contudo, observa-se uma diferença operacional: enquanto a média nacional geral registra um PUE de 1,45 (projetado para 1,36 em 2030), instalações dedicadas a cargas de IA impulsionadas pelo resfriamento líquido alcançam um PUE médio estimado de 1,145 (caindo para 1,136 até 2030) [21]. No cenário brasileiro, essa pressão energética reflete-se nas projeções

do Operador Nacional do Sistema Elétrico (ONS), CCEE e EPE, que contabilizam 27 contratos assinados e 30 pareceres favoráveis de acesso à Rede Básica até julho de 2026 (além de 21 em análise), projetando que a carga exclusiva de data centers no Sistema Interligado Nacional (SIN) salte de 315 MW médios em 2026 para 5.653 MW médios até 2030 [22]. Como essas novas instalações operam com densidades térmicas que saltaram de 10 kW para mais de 40 a 100 kW por *rack*, o resfriamento a ar tradicional torna-se termodinamicamente insuficiente, tornando obrigatória a combinação de refrigeração líquida direta ao chip com a consolidação agressiva de contêineres para mitigar o desperdício em megawatts de potência ociosa.

## REFERÊNCIAS


[1] Peter Mell and Timothy Grance. The NIST definition of cloud computing. Technical Report NIST Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, 2011.

[2] Fang Liu, Jin Tong, Jian Mao, Robert Bohn, John Messina, Lee Badger, and Dawn Leaf. NIST cloud computing reference architecture. Technical Report NIST Special Publication 500-292, National Institute of Standards and Technology, Gaithersburg, MD, 2011.

[3] Lee Badger, Timothy Grance, Robert Patt-Corner, and Jeff Voas. Cloud computing synopsis and recommendations. Technical Report NIST Special Publication 800-146, National Institute of Standards and Technology, Gaithersburg, MD, 2012.

[4] Karen Scarfone, Murugiah Souppaya, and Paul Hoffman. Guide to security for full virtualization technologies. Technical Report NIST Special Publication 800-125, National Institute of Standards and Technology, Gaithersburg, MD, 2011.

[5] OpenStack Foundation. Openstack documentation: Logical architecture, compute (Nova), networking (Neutron), identity (Keystone) and image (Glance) services. Documentação oficial, 2026. Disponível em: `https://docs.openstack.org/`.

[6] Murugiah Souppaya, John Morello, and Karen Scarfone. Application container security guide. Technical Report NIST Special Publication 800-190, National Institute of Standards and Technology, Gaithersburg, MD, 2017.

[7] Docker, Inc. Docker engine documentation: Architecture, storage drivers and networking. Documentação oficial, 2026. Disponível em: `https://docs.docker.com/engine/`.

[8] Liz Rice. *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, Sebastopol, CA, 2020.

[9] Sari Sultan, Imtiaz Ahmad, and Tassos Dimitrova. Container security: Issues, challenges, and the road ahead. *IEEE Access*, 7:52976–52996, 2019.

[10] Theo Combe, Antony Martin, and Roberto Di Pietro. To docker or not to docker: A security perspective. *IEEE Cloud Computing*, 3(5):54–62, 2016.

[11] Andrew Martin and Michael Hausenblas. *Hacking Kubernetes: Threat-Driven Analysis and Defense*. O'Reilly Media, Sebastopol, CA, 2021.

[12] CISA. Supply chain compromise of third-party GitHub Action `tj-actions/changed-files` (CVE-2025-30066). Cybersecurity and Infrastructure

Security Agency Alert, 2025. National Vulnerability Database (NVD) / GitHub Security Advisory.

[13] GitGuardian. The state of secrets sprawl and CI/CD supply chain attacks: Analysis of the GhostAction campaign. Technical report, GitGuardian Security Research, 2026.

[14] GitHub. GitHub availability and incident post-mortem reports (2025–2026). GitHub Engineering and Status History, 2026.

[15] ISO/IEC. ISO/IEC 30134-2:2016 information technology – data centres – key performance indicators – part 2: PUE. Technical report, International Organization for Standardization, Geneva, 2016.

[16] Miyuru Dayarathna, Yonggang Wen, and Rui Fan. Data center energy consumption modeling: A survey. *IEEE Communications Surveys & Tutorials*, 18(1):732–794, 2016.

[17] Anton Beloglazov, Jemal Abawajy, and Rajkumar Buyya. Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. *Future Generation Computer Systems*, 28(5):755–768, 2012.

[18] ASHRAE. *Thermal Guidelines for Data Processing Environments*. American Society of Heating, Refrigerating and Air-Conditioning Engineers, Atlanta, GA, 5 edition, 2021.

[19] IEA. Energy and AI: World energy outlook special report on data centre electricity demand. Technical report, International Energy Agency, Paris, 2025.

[20] CSO. Data centres metered electricity consumption 2024–2025. Technical report, Central Statistics Office, Government of Ireland, Cork, 2025.

[21] Arman Shehabi, Sarah J. Smith, Alex Hubbard, and Alex Newkirk. 2024–2026 United States data center energy usage report update. Technical report, Lawrence Berkeley National Laboratory (LBNL), Berkeley, CA, 2026.

[22] ONS and EPE. Expansão da demanda de grandes cargas e Data Centers no sistema interligado nacional. Technical report, Operador Nacional do Sistema Elétrico e Empresa de Pesquisa Energética, Brasília, 2026.